Introduction to Retrieval-Augmented Generation

Retrieval-Augmented Generation, or RAG, combines a language model with an external knowledge store so that answers can be grounded in documents the model was never trained on. Instead of relying solely on parameters learned during pretraining, a RAG system retrieves relevant passages at query time and feeds them into the model's context window alongside the user's question.

The core pipeline has three stages: chunking the source documents into retrievable units, embedding those units into a vector space, and indexing the vectors so that a query can be matched against them quickly. Each stage introduces its own trade-offs, and a weak choice early in the pipeline, such as poor chunking, limits how good retrieval can ever be, no matter how good the embedding model or the index is downstream.

VECTOR DATABASES AND SIMILARITY SEARCH

A vector database stores embeddings and answers nearest-neighbor queries: given a query vector, return the stored vectors most similar to it under some distance metric, usually cosine similarity or dot product. At small scale, brute-force search that compares the query against every stored vector is fast enough and perfectly exact.

Once a collection grows past a few hundred thousand vectors, brute-force scanning becomes too slow for interactive use, which is where approximate nearest-neighbor structures like HNSW come in. HNSW builds a multi-layer graph over the vectors so that a query can route through long-range links at the top layers before narrowing down to short-range links near the bottom, trading a small amount of recall for a large speedup over brute force.

Most production vector databases expose tunable parameters such as M, ef_construction, and ef_search that let you trade index build time, memory, and query latency against recall. Choosing those parameters is itself an empirical exercise, not a one-size-fits-all default.

Embedding Models for Semantic Search

An embedding model maps a piece of text to a fixed-length vector such that semantically similar pieces of text end up close together in that vector space. OpenAI's text-embedding-3-small is a common default choice: it is inexpensive, fast, and produces 1536-dimensional vectors that work well for general-purpose retrieval.

Larger embedding models, such as text-embedding-3-large, trade higher cost and latency for modestly better retrieval quality, which only matters once the cheaper model is demonstrably the bottleneck. It is usually better to start with the small model, measure recall on your own corpus, and only upgrade if the numbers say you need to.

Embedding quality is corpus-dependent. A model trained mostly on web text may underperform on dense legal or medical documents, so it is worth validating retrieval quality on a representative sample of test queries before committing to a model in production.

CHUNKING STRATEGIES FOR DOCUMENT PROCESSING

Fixed-size chunking splits a document by raw character count, sliding a window forward with some overlap between consecutive chunks. It is the simplest strategy to implement and the fastest to run, but it is completely blind to content, so it routinely cuts a chunk off in the middle of a sentence or even mid-word.

Recursive chunking tries a list of separators from coarsest to finest, such as paragraph breaks, then line breaks, then sentence boundaries, then word boundaries, and only falls back to a hard character cut when nothing else fits within the size limit. This respects natural text boundaries far more often than fixed-size splitting while still being cheap to compute.

Semantic chunking goes a step further: it embeds individual sentences and inserts a chunk boundary wherever the similarity between consecutive sentences drops below a threshold, on the theory that a similarity drop marks a topic shift. It produces the most topically coherent chunks of any strategy here, at the cost of one embedding call per sentence during indexing, which makes it considerably slower to build than the others.

Evaluating Retrieval Quality

Recall at k measures, out of the truly relevant documents for a query, what fraction were actually returned in the top k results. Precision at k measures the opposite direction: of the k results returned, what fraction were actually relevant. The two metrics trade off against each other as k changes, which is why teams usually report both rather than either alone.

Mean reciprocal rank rewards systems that place the first relevant result near the top of the ranked list, while normalized discounted cumulative gain, NDCG, accounts for graded relevance and the full ranking rather than just whether a relevant item appears at all. Choosing between them depends on whether your downstream use case cares about one best answer or a well-ordered list of several.

In practice, building even a small labeled set of query-to-relevant-chunk pairs and computing these metrics before and after a chunking or embedding change is the only reliable way to know whether the change actually helped, since eyeballing a handful of retrieved passages is easy to fool yourself with.

PRODUCTION DEPLOYMENT AND COST CONSIDERATIONS

Every chunk indexed costs one embedding call at index time and contributes tokens to the context window at query time, so the number and size of chunks directly drives both indexing cost and per-query cost. Smaller chunks generally improve retrieval precision but increase the chunk count, and therefore the token bill, for the same corpus.

Latency budgets matter just as much as cost. Semantic chunking's per-sentence embedding calls make index builds considerably slower than fixed, recursive, or document-aware chunking, which is a one-time cost at ingest

time, not a per-query cost, but it still matters for how quickly new documents become searchable.

Monitoring retrieval quality in production, not just at launch, catches drift as a corpus grows or shifts topic over time. A chunking strategy that worked well on the initial document set can quietly degrade as new, differently structured documents are added, which is why recall and precision dashboards belong alongside the usual latency and error-rate metrics.